DOCUMENT 17 — SECURITY, PRIVACY & APPLICATION SECURITY SPECIFICATION

1. Dokümanın Amacı

Bu dokümanın amacı, Smart Chef & Pantry AI uygulamasının Flutter mobil istemcisi, Flask backend servisi ve PostgreSQL veritabanı arasındaki iletişimde uygulanacak güvenlik ve gizlilik kurallarını tanımlamaktır.

Bu doküman özellikle aşağıdaki konuları kapsar:

- Kimlik doğrulama güvenliği
- Yetkilendirme
- JWT güvenliği
- Google Social Login güvenliği
- API güvenliği
- Kullanıcı verilerinin korunması
- PostgreSQL ve ORM güvenliği
- Flutter tarafında güvenli veri saklama
- Girdi doğrulama ve saldırı önleme
- Admin güvenliği
- Log ve hata mesajlarında veri sızıntısının önlenmesi
- HTTPS ve network güvenliği
- Rate limiting
- Güvenlik testleri
- MVP için minimum güvenlik gereksinimleri

Bu doküman Document 10 — Authentication & Authorization, Document 14 — Error Handling, Document 15 — Testing Strategy ve Document 16 — Deployment & DevOps dokümanlarının güvenlik açısından tamamlayıcısıdır.

---

2. Güvenlik Hedefleri

Smart Chef & Pantry AI için temel güvenlik hedefleri şunlardır:

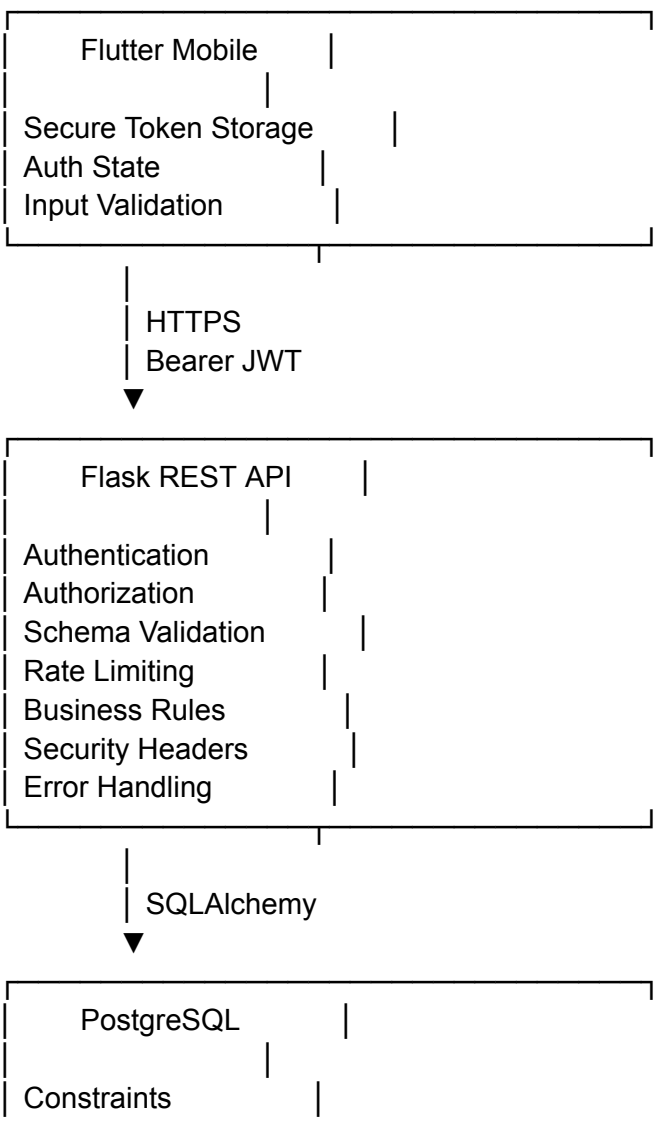
1. Kullanıcı hesabına yetkisiz erişimi engellemek.
2. Kullanıcının favorilerini başka kullanıcıların değiştirmesini engellemek.
3. Kullanıcının alışveriş sepetine başka kullanıcıların erişmesini engellemek.

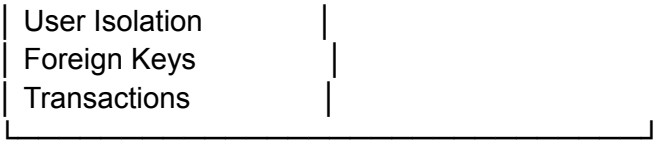
4. Google kimlik doğrulamasını backend tarafında doğrulamak.
5. JWT'nin güvenli şekilde üretilmesini, saklanmasını ve kullanılmasını sağlamak.
6. API üzerinden gelen kullanıcı girdilerini doğrulamak.
7. SQL Injection gibi saldırıları engellemek.
8. Hassas bilgilerin loglara yazılmasını engellemek.
9. Production ortamında HTTPS kullanmak.
10. Admin endpointlerinin yalnızca yetkili kullanıcılar tarafından kullanılmasını sağlamak.
11. Veritabanında kullanıcılar arası veri izolasyonu sağlamak.
12. Gereksiz kişisel verilerin toplanmasını önlemek.
13. Hatalı yapılandırma nedeniyle gizli bilgilerin açığa çıkmasını engellemek.

---

3. Güvenlik Mimarisi

Uygulamanın güvenlik mimarisi aşağıdaki şekilde olacaktır:





Temel güvenlik prensibi:

Flutter'a güvenme.
Client tarafından gönderilen kullanıcı kimliğine güvenme.
Client tarafından gönderilen role güvenme.
Backend bütün kritik kontrolleri kendisi yapmalıdır.

---

4. Güvenlik Katmanları

Uygulamada güvenlik tek bir katmanda uygulanmayacaktır.

Güvenlik aşağıdaki katmanlarda ele alınacaktır:

Layer 1 → Flutter Input Validation
Layer 2 → HTTPS / Network Security
Layer 3 → JWT Authentication
Layer 4 → Authorization
Layer 5 → API Schema Validation
Layer 6 → Business Rule Validation
Layer 7 → SQLAlchemy / ORM Security
Layer 8 → PostgreSQL Constraints
Layer 9 → Logging & Monitoring
Layer 10 → Deployment Security

Bir katmanda yapılan kontrol diğer katmanların yerine geçmez.

Örneğin:

Flutter quantity = 0 kontrolü

yapılmış olsa bile backend tekrar:

quantity > 0

kontrolünü yapmak zorundadır.

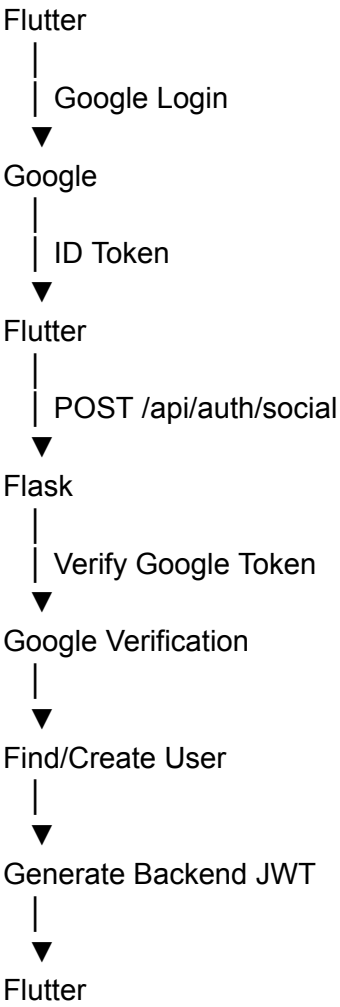
---

5. Kimlik Doğrulama Güvenliği

5.1 Google Social Login

MVP'de Google Social Login kullanılacaktır.

Akış:



Flutter'ın Google'dan aldığı token doğrudan güvenilir kabul edilmeyecektir.

Backend tokenı doğrulamalıdır.

---

6. Google Token Doğrulama

Backend aşağıdaki bilgileri kontrol etmelidir:

- Token signature
- Issuer
- Audience
- Expiration
- Subject
- Email bilgisi
- Token formatı

Kontrol başarısız olursa:

```
{
  "success": false,
  "error": {
    "code": "SOCIAL_AUTH_FAILED",
    "message": "Social authentication failed."
  }
}
```

döndürülmelidir.

Backend doğrulaması yapılmadan kullanıcı hesabı oluşturulmamalıdır.

---

7. JWT Güvenliği

Backend, başarılı social login sonrasında kendi JWT tokenını oluşturacaktır.

JWT minimum gerekli bilgiyi içermelidir.

Örnek:

```
{
  "sub": "user_uuid",
  "role": "user"
}
```

Gereksiz bilgiler JWT içine eklenmemelidir.

Özellikle:

password
email
profile details
private application data

gibi gereksiz bilgilerin token payload'ında tutulmasından kaçınılmalıdır.

---

8. JWT Secret Güvenliği

JWT secret kesinlikle source code içerisinde tutulmayacaktır.

Yanlış:

JWT_SECRET_KEY = "my-secret-key"

Doğru yaklaşım:

Environment Variable
↓
JWT_SECRET_KEY
↓
Flask Configuration
↓
JWT Extension

`.env`:

JWT_SECRET_KEY=very-long-random-secret

`.env` Git repository'ye gönderilmeyecektir.

`.gitignore`:

.env
.env.*
!.env.example

---

9. JWT Süresi

MVP'de access token kullanılacaktır.

Örneğin:

JWT_ACCESS_TOKEN_EXPIRES

environment variable üzerinden yönetilebilir.

Token süresi dolduğunda backend:

401 Unauthorized
TOKEN_EXPIRED

döndürmelidir.

Flutter:

TOKEN_EXPIRED
↓
Clear Token
↓
Unauthenticated
↓
Login Screen

akışını uygulamalıdır.

---

10. Refresh Token

MVP kapsamında refresh token zorunlu değildir.

İlk sürüm:

Login
↓
Access JWT
↓
API Requests
↓
Expiration
↓
Login Again

şeklinde çalışabilir.

İlerleyen sürümlerde:

Access Token
+
Refresh Token

mekanizması eklenebilir.

Refresh token uygulanacaksa ayrıca:

- rotation
- revocation
- secure storage
- expiration
- reuse detection

gibi mekanizmalar değerlendirilmelidir.

---

11. Flutter Token Saklama

JWT Flutter tarafında plain text olarak:

SharedPreferences

içerisinde tutulmamalıdır.

Token için secure storage tercih edilmelidir.

Örneğin:

flutter_secure_storage

kullanılabilir.

Yapı:

Flutter
↓
Secure Storage
↓
JWT

Token:

print()

debug console
logs
analytics

üzerinden gösterilmemelidir.

---

12. Authorization

Authentication:

"Bu kullanıcı kim?"

sorusuna cevap verir.

Authorization:

"Bu kullanıcı bu işlemi yapabilir mi?"

sorusuna cevap verir.

Örneğin:

GET /api/favorites

için kullanıcı authenticated olmalıdır.

Ancak:

DELETE /api/favorites/{recipe_id}

işleminde sadece authenticated olmak yeterli değildir.

Backend ilgili favorite kaydının gerçekten mevcut kullanıcıya ait olduğunu kontrol etmelidir.

---

13. Kullanıcı Kimliğinin Güvenli Kullanımı

En önemli güvenlik kurallarından biri:

Client'tan gelen:

```
{
  "user_id": "..."
}
```

bilgisine güvenilmemelidir.

Örneğin:

```
POST /api/favorites
Authorization: Bearer JWT
```

JWT içerisindeki:

sub

değeri backend tarafından okunmalıdır.

```
JWT.sub
  ↓
current_user_id
  ↓
Database Query
```

Böylece kullanıcı başka bir kullanıcının ID'sini göndererek işlem yapamaz.

---

14. IDOR Koruması

IDOR (Insecure Direct Object Reference), bir kullanıcının başka kullanıcıya ait kaynağa doğrudan ID üzerinden erişebilmesi problemidir.

Örneğin saldırgan:

```
GET /api/cart/items/123
```

gönderdiğinde backend sadece:

cart_item.id = 123

kontrolü yapmamalıdır.

Şu kontrol yapılmalıdır:

```
cart_item.id = requested_id
AND
cart_item.user_id = current_user_id
```

Aynı kural:

- Favorites
- Cart
- User data
- Future private resources

için uygulanmalıdır.

---

15. Admin Güvenliği

Admin endpointleri:

/api/admin/*

altında bulunacaktır.

Admin kontrolü yalnızca Flutter tarafından yapılmayacaktır.

Yanlış yaklaşım:

```
if (user.role == "admin") {
  showAdminPanel();
}
```

Bu yalnızca UI kontrolüdür.

Gerçek güvenlik backend'de olmalıdır.

Backend:

```
JWT
 ↓
current_user
 ↓
role == admin?
```

↓
Yes → Continue
No → 403

uygulamalıdır.

---

16. Admin Endpoint Güvenlik Akışı

Örneğin:

POST /api/admin/recipes

isteğinde:

1. JWT var mı?
2. JWT geçerli mi?
3. Kullanıcı mevcut mu?
4. Kullanıcının role değeri admin mi?
5. Request body geçerli mi?
6. Business validation başarılı mı?
7. Database transaction
8. Response

şeklinde ilerlenmelidir.

Yetkisiz kullanıcı:

```
{
  "success": false,
  "error": {
    "code": "ADMIN_REQUIRED",
    "message": "Admin access is required."
  }
}
```

almalıdır.

HTTP:

403 Forbidden

olmalıdır.

---

17. API Input Validation

Backend hiçbir zaman client'ın gönderdiği veriyi doğrudan database'e yazmamalıdır.

Örneğin cart request:

```
{
  "ingredient_id": "...",
  "quantity": 2
}
```

için:

ingredient_id → UUID mi?
ingredient mevcut mu?
quantity → numeric mi?
quantity > 0 mı?
unit → geçerli formatta mı?

kontrolleri yapılmalıdır.

---

18. SQL Injection Koruması

SQL sorguları string birleştirme yöntemiyle oluşturulmamalıdır.

Yanlış:

```
query = f"SELECT * FROM ingredients WHERE name = '{name}'"
```

Bunun yerine SQLAlchemy ORM / parameterized query kullanılmalıdır.

Örneğin:

```
Ingredient.query.filter(
    Ingredient.normalized_name == normalized_name
).all()
```

kullanılmalıdır.

Böylece kullanıcı girdisinin SQL komutuna dönüşmesi engellenir.

---

19. XSS ve HTML İçerikleri

Smart Chef & Pantry AI'nin temel verileri JSON üzerinden çalışmaktadır.

Recipe:

title
description
instructions

gibi alanlar kullanıcı tarafından oluşturuluyorsa veya dış kaynaktan alınıyorsa güvenli şekilde işlenmelidir.

Recipe içeriği Flutter UI içerisinde doğrudan HTML olarak çalıştırılmamalıdır.

Eğer ileride HTML içerik gösterilecekse:

HTML sanitization
trusted source validation
script filtering

uygulanmalıdır.

---

20. URL Güvenliği

Recipe kaynakları:

source_url
youtube_url

alanlarında tutulacaktır.

Backend URL formatını doğrulamalıdır.

Özellikle YouTube URL'si için yalnızca desteklenen domainlerin kabul edilmesi önerilir.

Örneğin:

youtube.com
www.youtube.com
youtu.be

gibi domainler kontrollü şekilde desteklenebilir.

Flutter tarafında URL:

url_launcher

ile açılmadan önce de temel format kontrolünden geçirilebilir.

---

21. SSRF Koruması

MVP'de backend'in kullanıcı tarafından verilen URL'lere server-side HTTP request göndermesi planlanmamaktadır.

Bu nedenle:

Backend → User supplied URL → HTTP Request

akışı kullanılmamalıdır.

İleride recipe import veya external content fetch özelliği eklenirse SSRF koruması ayrıca tasarlanmalıdır.

Özellikle:

localhost
127.0.0.1
private IP ranges
internal services
cloud metadata endpoints

gibi adreslere erişim engellenmelidir.

---

22. CORS Güvenliği

Development ortamında:

localhost

gibi kaynaklar kullanılabilir.

Production ortamında wildcard:

*

kullanılmamalıdır.

Production:

CORS_ORIGINS=https://trusted-domain.example

gibi kontrollü kaynaklarla sınırlandırılmalıdır.

Mobil uygulama mimarisinde CORS gereksinimi platforma göre farklılık gösterebilir; yine de backend'e ileride web client eklenmesi ihtimaline karşı production policy açıkça tanımlanmalıdır.

---

23. HTTPS

Production ortamında API yalnızca HTTPS üzerinden erişilebilir olmalıdır.

Yanlış:

http://api.example.com

Doğru:

https://api.example.com

HTTPS:

Flutter
  ↓ encrypted
Internet
  ↓ encrypted
Flask API

şeklinde network trafiğini korur.

---

24. Certificate Validation

Flutter production uygulamasında TLS certificate validation devre dışı bırakılmamalıdır.

Özellikle aşağıdaki tarz uygulamalar production'da kullanılmamalıdır:

badCertificateCallback = (_, __, ___) => true;

Bu tür bir yapı yalnızca kontrollü local development/debug ortamlarında dahi dikkatle kullanılmalıdır.

Production build'de kesinlikle bulunmamalıdır.

---

25. Rate Limiting

Public ve authentication endpointleri abuse'a karşı rate limit ile korunmalıdır.

Özellikle:

POST /api/auth/social
POST /api/recipes/recommend
GET /api/ingredients

endpointleri önemlidir.

Örneğin kavramsal olarak:

Authentication:
limited requests / minute / IP

Recommendation:
limited requests / minute / user

Ingredient Search:
limited requests / minute / IP

şeklinde politika uygulanabilir.

Kesin limitler production trafik ölçümleri sonrasında belirlenmelidir.

---

26. Brute Force ve Abuse Koruması

Google Social Login kullanıldığı için klasik password brute-force saldırısı MVP'de bulunmamaktadır.

Bununla birlikte:

auth endpoint abuse
token abuse
recommendation spam
API scraping

gibi saldırılara karşı rate limiting uygulanmalıdır.

---

27. Database Security

PostgreSQL production ortamında doğrudan internet üzerinden erişilebilir olmamalıdır.

Önerilen yapı:

Internet
↓
Reverse Proxy / API
↓
Flask
↓
Private PostgreSQL Network
↓
PostgreSQL

Database portu yalnızca gerekli servislerden erişilebilir olmalıdır.

---

28. Database User Permissions

Production uygulaması PostgreSQL'e superuser olarak bağlanmamalıdır.

Uygulama için ayrı database user kullanılmalıdır.

Örneğin:

smartchef_app

kullanıcısının yalnızca uygulamanın ihtiyaç duyduğu database erişimlerine sahip olması tercih edilir.

---

29. Database Constraints

Application-level validation yeterli değildir.

Database seviyesinde de:

PRIMARY KEY
FOREIGN KEY
UNIQUE
NOT NULL
CHECK

kısıtları kullanılmalıdır.

Örneğin:

cart quantity > 0

gibi kurallar mümkün olduğunda database seviyesinde de desteklenebilir.

---

30. Favorite Güvenliği

Favorite kaydı:

user_id
recipe_id

ile ilişkilidir.

Database:

UNIQUE(user_id, recipe_id)

kısıtını içermelidir.

Böylece aynı recipe'nin aynı kullanıcı tarafından birden fazla favorite olarak eklenmesi engellenir.

Backend ayrıca duplicate işlemini kontrol etmelidir.

---

31. Cart Güvenliği

Cart item:

user_id
ingredient_id
quantity
unit

ile ilişkilidir.

Her cart sorgusunda:

current_user_id

kullanılmalıdır.

Başka kullanıcının cart item ID'si bilinse bile veri döndürülmemelidir.

Aynı şekilde update/delete işlemlerinde ownership kontrolü yapılmalıdır.

---

32. Sensitive Data Minimization

MVP uygulaması gereksiz kişisel veri toplamamalıdır.

Gerekli olabilecek kullanıcı bilgileri:

provider
provider_user_id

name
email
profile_image_url
created_at
updated_at

şeklindedir.

Aşağıdaki türde bilgilerin gereksiz yere toplanmaması gerekir:

Contact list
Exact location
Device contacts
Unnecessary personal identifiers
Payment information
Password

MVP kapsamında payment information zaten tutulmayacaktır.

---

33. Email Güvenliği

Google Social Login üzerinden elde edilen email backend tarafından doğrulanmalıdır.

Email bilgisi:

normalized / validated

şekilde işlenmelidir.

Kullanıcı email adresi response içerisinde gerekiyorsa gönderilebilir; gerekmiyorsa her endpointte tekrar gönderilmesine gerek yoktur.

---

34. Logging Güvenliği

Loglar debugging ve monitoring için kullanılacaktır.

Ancak aşağıdaki bilgiler loglanmamalıdır:

JWT
Google ID Token

JWT Secret
Database Password
Sensitive personal data
Authorization Header

Yanlış:

Authorization: Bearer eyJ...

Doğru:

Authorization: [REDACTED]

---

35. Request ID

Her request için mümkünse unique request ID kullanılmalıdır.

Örneğin:

X-Request-ID

veya backend tarafından üretilen UUID.

Örnek:

request_id = 8d6c...

Log:

ERROR
request_id=8d6c...
endpoint=/api/recipes/recommend
error=DATABASE_ERROR

Kullanıcıya:

```
{
  "success": false,
  "error": {
    "code": "INTERNAL_SERVER_ERROR",
    "message": "An unexpected error occurred.",
```

```
    "details": {
      "request_id": "8d6c..."
    }
  }
}
```

gibi kontrollü bilgi verilebilir.

---

36. Production Error Security

Production ortamında exception'ın tamamı kullanıcıya gönderilmemelidir.

Yanlış:

```
{
  "error": "psycopg2.errors.UniqueViolation: ..."
}
```

Doğru:

```
{
  "success": false,
  "error": {
    "code": "DATABASE_ERROR",
    "message": "A database error occurred."
  }
}
```

Detay backend loglarında tutulmalıdır.

---

37. Debug Mode

Production'da:

DEBUG = False

olmalıdır.

Flask debug ekranı production ortamında kesinlikle açık bırakılmamalıdır.

---

38. Environment Security

Environment'lar birbirinden ayrılmalıdır:

Development
Testing
Production

Örneğin development database:

smartchef_dev

testing:

smartchef_test

production:

smartchef_prod

olabilir.

Testlerin production database üzerinde çalıştırılması yasaktır.

---

39. Secret Management

Aşağıdaki değerler secret olarak değerlendirilmelidir:

JWT_SECRET_KEY
DATABASE_PASSWORD
GOOGLE_CLIENT_SECRET
API keys
Cloud credentials

Bunlar:

GitHub
GitLab
public repository
Flutter source
README
screenshots

içerisinde paylaşılmamalıdır.

---

40. `.env.example`

Repository'de yalnızca örnek yapı bulunmalıdır:

```
DATABASE_URL=
JWT_SECRET_KEY=
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
FLASK_ENV=development
```

Gerçek secret değerleri:

```
.env
```

veya production secret management sistemi içerisinde tutulmalıdır.

---

41. Git Güvenliği

Commit edilmemesi gerekenler:

```
.env
*.pem
*.key
credentials.json
service-account.json
production secrets
database dumps containing private data
```

`.gitignore` buna göre hazırlanmalıdır.

---

42. Secret Sızıntısı Durumunda

Eğer bir secret yanlışlıkla Git repository'ye gönderilirse:

1. Secret hemen revoke/rotate edilir.
2. Yeni secret oluşturulur.
3. Production configuration güncellenir.
4. Eski secret geçersiz hale getirilir.
5. Repository history incelenir.
6. Gerekiyorsa ilgili kullanıcı/servis güvenlik kayıtları incelenir.

Sadece GitHub'dan dosyayı silmek yeterli güvenlik önlemi değildir.

---

43. Flutter Secure Coding

Flutter tarafında:

- Tokenlar secure storage'da tutulmalı.
- Secret key uygulama içine gömülmemeli.
- Backend secretları Flutter'a konulmamalı.
- API URL'leri environment/config üzerinden yönetilmeli.
- Debug logları production build'de azaltılmalı.
- Kullanıcı girdileri kontrol edilmeli.
- Authentication state merkezi yönetilmeli.

Özellikle:

GOOGLE_CLIENT_SECRET
JWT_SECRET_KEY
DATABASE_PASSWORD

Flutter uygulamasında bulunmamalıdır.

---

44. API Security Rules

Her endpoint için aşağıdaki sorular cevaplanmalıdır:

Authentication gerekli mi?
Authorization gerekli mi?
Input validation var mı?
Resource ownership kontrol ediliyor mu?
Rate limit gerekli mi?
Sensitive data döndürülüyor mu?
Error response güvenli mi?
Logging güvenli mi?

Endpoint geliştirme sırasında bu kontrol listesi kullanılacaktır.

---

45. Endpoint Güvenlik Matrisi

Endpoint	Auth	Authorization	Validation	Ownership
POST `/api/auth/social`	Hayır	Hayır	Evet	Hayır
GET `/api/me`	Evet	User	Evet	JWT
GET `/api/ingredients`	Hayır	Hayır	Evet	Hayır
POST `/api/recipes/recommend`	Hayır	Hayır	Evet	Hayır
GET `/api/recipes/{id}`	Hayır	Hayır	UUID	Hayır
GET `/api/favorites`	Evet	User	Evet	JWT
POST `/api/favorites`	Evet	User	Evet	JWT
DELETE `/api/favorites/{id}`	Evet	User	Evet	User ownership
GET `/api/cart`	Evet	User	Evet	JWT
POST `/api/cart/items`	Evet	User	Evet	JWT
PATCH `/api/cart/items/{id}`	Evet	User	Evet	User ownership
DELETE `/api/cart/items/{id}`	Evet	User	Evet	User ownership
POST `/api/admin/recipes`	Evet	Admin	Evet	Admin

PUT `/api/admin/recipes/{id}`	Evet	Admin	Evet	Admin
DELETE `/api/admin/recipes/{id}`	Evet	Admin	Evet	Admin

---

46. Recipe Recommendation Security

Recommendation endpointi public olsa bile sınırsız ve doğrulamasız bırakılmamalıdır.

Request:

```
{
  "ingredient_ids": [
    "uuid-1",
    "uuid-2"
  ]
}
```

için:

ingredient_ids list mi?
Boş mu?
UUID'ler geçerli mi?
Duplicate var mı?
Ingredient gerçekten mevcut mu?
Maximum item limit aşılıyor mu?

kontrolleri yapılmalıdır.

Örneğin çok büyük bir liste gönderilerek backend'in gereksiz kaynak tüketmesi engellenmelidir.

---

47. Denial of Service Risklerinin Azaltılması

MVP seviyesinde temel önlemler:

Request body size limit

Pagination
Rate limiting
Maximum ingredient selection limit
Maximum cart quantity
Query timeout
Database indexes

kullanılmalıdır.

Özellikle pagination olmayan büyük veri endpointlerinden kaçınılmalıdır.

---

48. Pagination Güvenliği

Ingredient listesi:

GET /api/ingredients?page=1&limit=20

şeklinde kullanılmalıdır.

Backend:

limit <= MAX_PAGE_SIZE

kontrolü yapmalıdır.

Örneğin:

limit=1000000

gibi requestler kabul edilmemelidir.

---

49. Mass Assignment Koruması

Client'ın gönderdiği bütün alanlar otomatik olarak database modeline aktarılmamalıdır.

Örneğin kullanıcı requestinde:

```
{
  "name": "Esma",
```

```
  "role": "admin"
}
```

gönderirse backend bunu doğrudan model update'e çevirmemelidir.

Kullanıcı tarafından değiştirilebilir alanlar açıkça tanımlanmalıdır.

Örneğin:

Allowed:
name
profile_image_url

Not allowed:
role
provider
provider_user_id
created_at

---

50. Authorization by Design

Authorization sonradan eklenen bir kontrol olmamalıdır.

Her protected feature geliştirilirken:

Who?
What?
Which resource?
What ownership?
Which role?

soruları cevaplanmalıdır.

Örneğin:

User A
↓
Favorite Recipe
↓
Recipe 10

User B:

Recipe 10
↓
User A's Favorite
↓
DENY

---

51. WebView Güvenliği

MVP'de mümkün olduğunca:

url_launcher

ile external browser kullanılacaktır.

WebView yalnızca uygulama içinde gösterilmesi gerçekten gerekli olan güvenilir içerikler için kullanılmalıdır.

WebView kullanılırsa:

- yalnızca HTTPS
- güvenilir domain
- gereksiz JavaScript kapalı
- navigation kontrolü
- external link kontrolü

uygulanmalıdır.

---

52. Deep Link Güvenliği

İleride recipe deep link desteği eklenirse kullanıcıdan gelen deep link parametreleri güvenilir kabul edilmemelidir.

Örneğin:

smartchef://recipe/123

geldiğinde backend/API üzerinden recipe ID doğrulanmalıdır.

---

53. Dependency Security

Hem Flutter hem Python bağımlılıkları düzenli olarak güncellenmelidir.

Kontrol edilmesi gerekenler:

Flutter packages
Dart packages
Python packages
Flask
SQLAlchemy
JWT library
PostgreSQL driver

Güncellemeler yapılmadan önce test suite çalıştırılmalıdır.

Amaç:

Security Update
↓
Dependency Update
↓
Tests
↓
Deploy

akışını sağlamaktır.

---

54. Dependency Locking

Dependency sürümleri mümkün olduğunca lock dosyalarıyla kontrol altında tutulmalıdır.

Flutter:

pubspec.lock

Backend:

requirements.txt

veya uygun dependency lock mekanizması.

Production'da test edilmemiş dependency sürümlerinin otomatik olarak alınmasından kaçınılmalıdır.

---

55. Security Testing

Document 15'te tanımlanan testlere ek olarak güvenlik testleri uygulanacaktır.

Temel testler:

Invalid JWT
Expired JWT
Missing JWT
Invalid Google token
Non-admin → Admin endpoint
User A → User B favorite
User A → User B cart
Invalid UUID
Negative quantity
Huge quantity
Huge pagination limit
Malformed JSON
Unexpected fields
SQL injection payload
Invalid URL
Rate limit

---

56. SQL Injection Test Örnekleri

Ingredient search gibi alanlarda güvenlik testi yapılmalıdır.

Örneğin:

'
"
' OR '1'='1

gibi payloadlar gönderildiğinde uygulama:

500 Internal Server Error

veya SQL syntax error vermemelidir.

Beklenen davranış:

Normal validation / empty result / controlled response

olmalıdır.

---

57. Authorization Testleri

Test senaryosu:

User A login
User B login

User A creates favorite

User B attempts:
DELETE /api/favorites/{User A favorite}

Beklenen:

403 Forbidden

veya güvenlik tasarımına uygun kontrollü bir kaynak bulunamadı yanıtıdır.

Aynı test cart için de yapılmalıdır.

---

58. Authentication Testleri

Minimum testler:

Valid Google token
Invalid Google token
Expired Google token
Wrong audience
Wrong issuer
Missing token
Malformed JWT

Expired JWT
Tampered JWT

JWT payload değiştirilerek imza geçersiz hale getirildiğinde backend tokenı kabul etmemelidir.

---

59. Admin Security Tests

Test:

Normal User
↓
POST /api/admin/recipes

Beklenen:

403 Forbidden
ADMIN_REQUIRED

Admin:

Admin
↓
POST /api/admin/recipes
↓
Validation
↓
Create

şeklinde başarılı olabilir.

---

60. Privacy-Oriented API Responses

API response'ları minimum gerekli veriyi döndürmelidir.

Örneğin `/api/me`:

{

```
  "id": "uuid",
  "name": "User",
  "email": "user@example.com",
  "profile_image_url": "..."
}
```

gibi gerekli alanları içerebilir.

Backend internal alanları response'a eklenmemelidir.

Örneğin:

database internal metadata
security secrets
internal flags
password-related fields

döndürülmemelidir.

---

61. Database Backup Security

Production database backupları da uygulama verisi kadar korunmalıdır.

Backup:

Encrypted
Access controlled
Retention policy
Separate storage

ile korunmalıdır.

Backup dosyaları public repository'de tutulmamalıdır.

---

62. Backup Restore Testi

Backup oluşturmak tek başına yeterli değildir.

Periyodik olarak:

Backup
↓
Restore Test
↓
Database Integrity Check
↓
Application Connection

test edilmelidir.

Amaç gerçek bir problem durumunda backup'ın kullanılabilir olduğundan emin olmaktır.

---

63. Security Headers

Backend/reverse proxy seviyesinde uygun HTTP security headers kullanılmalıdır.

Örneğin web erişimi söz konusu olduğunda:

Strict-Transport-Security
X-Content-Type-Options
Content-Security-Policy
Referrer-Policy

gibi headerlar değerlendirilebilir.

Mobil API olması nedeniyle bütün headerlar aynı önemde değildir; deployment mimarisine göre uygulanmalıdır.

---

64. File Upload Güvenliği

MVP'de kullanıcı tarafından server'a dosya yükleme özelliği bulunmamaktadır.

İleride:

Profile image upload
Recipe image upload

eklenirse ayrıca:

File type validation

File size limit
Extension validation
Content validation
Random file name
Storage isolation
Malware scanning

gibi kontroller uygulanmalıdır.

---

65. External Recipe Content

Recipe source URL'leri uygulama tarafından güvenilir kaynaklardan oluşturulmalıdır.

MVP'de uncontrolled web scraping yapılmayacaktır.

Admin tarafından eklenen:

source_url
youtube_url

değerleri validation'dan geçirilmelidir.

Böylece kullanıcıların zararlı veya beklenmeyen URL'lerle yönlendirilmesi riski azaltılır.

---

66. Mobile Application Security

Flutter APK/AAB içerisinde:

API Secret
Database Password
JWT Secret
Google Client Secret

bulundurulmayacaktır.

Client tarafında bulunan değerlerin kullanıcı tarafından reverse engineering ile görülebileceği varsayılmalıdır.

Bu nedenle gerçek secret yalnızca backend tarafında bulunmalıdır.

---

67. Obfuscation

Release build'de Flutter'ın desteklediği kod obfuscation seçenekleri gerektiğinde değerlendirilebilir.

Ancak obfuscation:

Secret management
Authentication
Authorization

yerine geçmez.

Güvenliğin temel mekanizması olarak görülmemelidir.

---

68. Session Security

MVP stateless JWT mimarisi kullandığı için server tarafında klasik session storage zorunlu değildir.

Kullanıcı logout olduğunda:

Flutter Secure Storage
↓
JWT deleted

yapılır.

JWT'nin kendisi expiration süresine kadar teknik olarak geçerli olabilir.

İleri sürümde token revocation / blacklist mekanizması değerlendirilebilir.

---

69. Account Lifecycle

Kullanıcı:

Google Login

↓
User Created

şeklinde oluşturulur.

Aynı provider ve provider user ID ile tekrar login:

Existing User

olarak bulunmalıdır.

Yeni duplicate kullanıcı oluşturulmamalıdır.

Database:

UNIQUE(provider, provider_user_id)

kısıtını taşımalıdır.

---

70. Security Incident Senaryoları

Senaryo 1 — JWT Sızıntısı

Token exposed
↓
Token expiration / revoke strategy
↓
User re-authentication
↓
Logs reviewed

Senaryo 2 — Database Credential Leak

Credential detected
↓
Credential rotation
↓
Database access review
↓
New credentials deployment
↓

Logs review

Senaryo 3 — Admin Account Compromise

Admin access suspicious
↓
Admin credential/session invalidation strategy
↓
Role/access review
↓
Logs
↓
Credential rotation

---

71. Security Monitoring

Production ortamında en azından aşağıdaki olaylar izlenmelidir:

Repeated authentication failures
Invalid JWT attempts
Admin endpoint failures
Rate limit violations
Database connection failures
500 errors
Unexpected request volume

Loglar kişisel verileri gereksiz yere içermemelidir.

---

72. Security Audit Checklist

Release öncesinde:

[ ] HTTPS aktif
[ ] DEBUG=False
[ ] JWT secret environment'tan geliyor
[ ] Database password source code'da yok
[ ] Google client secret source code'da yok
[ ] .env Git'te değil
[ ] JWT secure storage'da

[ ] Authorization backend'de kontrol ediliyor
[ ] Admin role backend'de kontrol ediliyor
[ ] Ownership kontrolleri mevcut
[ ] Input validation aktif
[ ] SQLAlchemy ORM kullanılıyor
[ ] Rate limiting aktif
[ ] Pagination limitleri mevcut
[ ] Production database public değil
[ ] Sensitive loglar temizlendi
[ ] Error response internal detay göstermiyor
[ ] Dependency security kontrolü yapıldı
[ ] Security tests geçti

---

73. MVP Security Priority

P0 — Zorunlu

HTTPS
JWT authentication
Google token verification
Backend authorization
Ownership checks
Admin authorization
Input validation
SQLAlchemy parameterized queries
Secure token storage
Secret management
Production DEBUG=False
Secure error handling
Basic rate limiting
Database access control

P1 — Önemli

Request ID
Security monitoring
Automated dependency scanning
Backup encryption
Restore testing
Security headers
Advanced rate limiting

P2 — Gelecek

Refresh token rotation
Advanced session revocation
Device/session management
Advanced anomaly detection
Automated security scanning pipeline
Advanced WAF

---

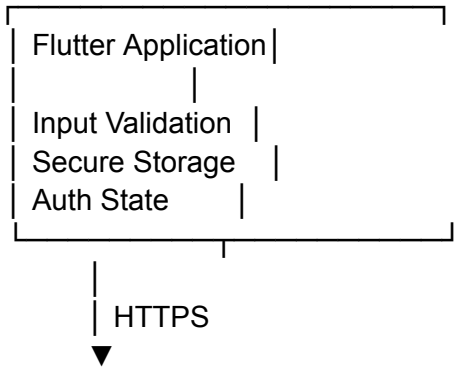
74. Güvenlik Anti-Pattern'leri

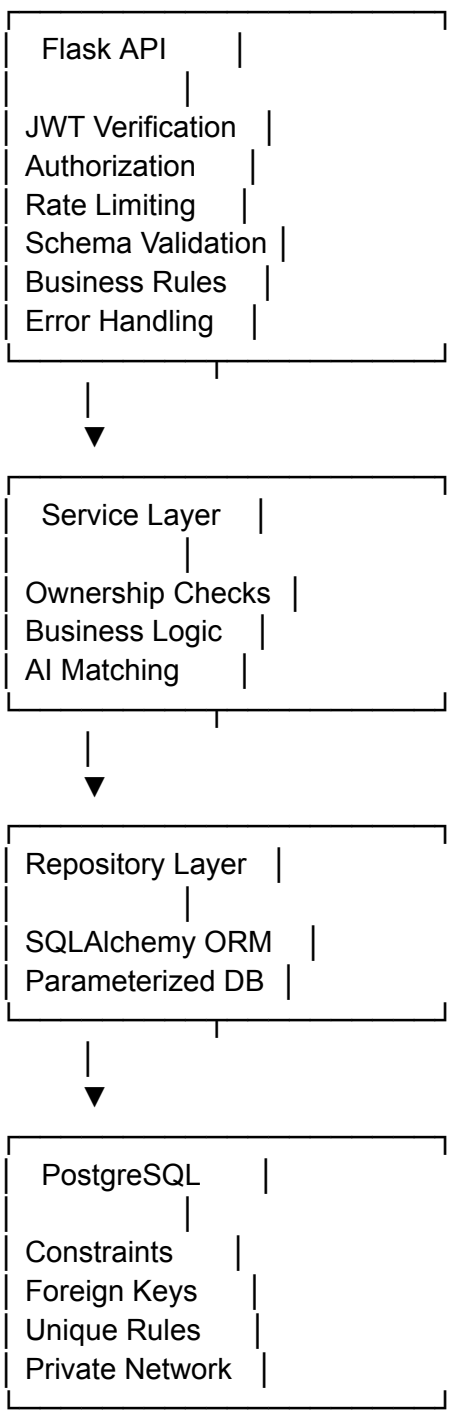
Aşağıdaki uygulamalar yapılmayacaktır:

❌ JWT secret'ı Flutter'a koymak
❌ Database password'u GitHub'a koymak
❌ Client user_id'sine güvenmek
❌ Client role değerine güvenmek
❌ Admin kontrolünü sadece UI'da yapmak
❌ SQL string concatenation kullanmak
❌ Tokenları loglamak
❌ Production'da DEBUG=True
❌ Production'da HTTP kullanmak
❌ User A'nın resource'una ownership kontrolü olmadan erişmek
❌ Sınırsız pagination
❌ Sınırsız request
❌ Internal exception detaylarını client'a göndermek
❌ .env dosyasını repository'ye göndermek
❌ Production database'i internete açık bırakmak

---

75. Güvenlik Mimarisi — Nihai Akış





---

76. Security Rules Summary

Smart Chef & Pantry AI için temel güvenlik kuralları:

1. Client güvenilir kaynak kabul edilmez.
2. Authentication backend tarafından doğrulanır.

3. Authorization backend tarafından yapılır.
4. Kullanıcı kimliği JWT'den alınır.
5. Client tarafından gönderilen `user_id` güvenilir kabul edilmez.
6. Kullanıcı kaynaklarında ownership kontrolü yapılır.
7. Admin yetkisi backend'de kontrol edilir.
8. JWT secure storage'da tutulur.
9. Secret değerler environment üzerinden yönetilir.
10. Production yalnızca HTTPS üzerinden çalışır.
11. SQLAlchemy/parameterized query kullanılır.
12. Tüm API girdileri backend'de doğrulanır.
13. Rate limiting uygulanır.
14. Production error response'ları internal detay içermez.
15. Sensitive bilgiler loglanmaz.
16. PostgreSQL private erişim arkasında tutulur.
17. Database constraints kullanılır.
18. Dependency güvenliği düzenli kontrol edilir.
19. Backuplar güvenli şekilde saklanır ve restore test edilir.
20. Security testleri release sürecinin parçasıdır.

---

77. Definition of Security Done

Bir özellik güvenli olarak tamamlanmış kabul edilmesi için:

[ ] Authentication gereksinimi belirlendi
[ ] Authorization gereksinimi belirlendi
[ ] Ownership kontrolü yapıldı
[ ] Input validation eklendi
[ ] API schema validation tamamlandı
[ ] Database constraints kontrol edildi
[ ] Sensitive data response'tan çıkarıldı
[ ] Error handling uygulandı
[ ] Logging güvenli
[ ] Rate limit değerlendirildi
[ ] Security testleri yazıldı
[ ] Normal kullanıcı ile test edildi
[ ] Yetkisiz kullanıcı ile test edildi
[ ] Admin senaryosu test edildi
[ ] Production configuration kontrol edildi

---

78. Document 17 Sonucu

Smart Chef & Pantry AI'nin güvenlik mimarisi Flutter, Flask ve PostgreSQL katmanlarının tamamını kapsayacak şekilde tasarlanmıştır.

MVP'de özellikle:

Authentication
Authorization
Ownership
JWT
Secure Storage
Input Validation
SQL Injection Protection
Admin Security
HTTPS
Rate Limiting
Secret Management
Database Security
Secure Error Handling
Logging Security

konuları temel güvenlik gereksinimleri olarak kabul edilmiştir.

Bu yapı sayesinde güvenlik yalnızca login ekranına bağlı olmayan, uygulamanın bütün katmanlarına dağıtılmış bir mimariye dönüşmektedir.

Bir sonraki geliştirme aşamasında bu dokümanda tanımlanan kurallar doğrudan backend, Flutter ve deployment kodlarına uygulanmalıdır.

END OF DOCUMENT 17